

Final Project: Implementation of Cryptographic Techniques for DataSafe Bank

In the final project, you will demonstrate your ability to protect DataSafe Bank's information by implementing cryptographic processes, including encryption and hashing.

Estimated duration: 90 minutes

This project consists of eight tasks:

- **Task 1: Rivest–Shamir–Adleman (RSA) key generation**
Generate RSA private and public keys to secure communications.
- **Task 2: RSA encryption and decryption**
Encrypt sensitive data using the RSA public key and decrypt it with the private key.
- **Task 3: Advanced Encryption Standard (AES) encryption**
Use AES-256 encryption for secure data storage and decrypt it back to verify accuracy.
- **Task 4: Digital signature creation**
Create and verify digital signatures to ensure document authenticity.
- **Task 5: Diffie-Hellman key exchange**
Establish secure shared secrets between two parties.
- **Task 6: Public Key Infrastructure (PKI) setup**
Create a root certificate, sign server certificates, and establish a trusted certificate authority (CA) hierarchy.
- **Task 7: Hashing with Secure Hash Algorithm 256 (SHA-256)**
Hash passwords to securely store them and verify their integrity.
- **Task 8: Digital certificates generation**
Generate certificate signing requests (CSRs) and self-signed certificates to issue secure communications credentials.

Refresh your knowledge

Before starting this project, you can refresh your knowledge by referring to the following labs:

- Module 1, Lesson 1: Cryptanalysis using CrypTool
- Module 1, Lesson 2: RSA using OpenSSL
- Module 1, Lesson 2: AES using OpenSSL
- Module 1, Lesson 2: Diffie-Hellman Using OpenSSL
- Module 2, Lesson 2: Lab: PKI Using OpenSSL
- Module 2, Lesson 3: Steganography using CrypTool
- Module 3, Lesson 1: Hashing Using OpenSSL
- Module 3, Lesson 1: Digital Signatures using OpenSSL
- Module 3, Lesson 1: Digital Certificate Using OpenSSL

Project scenario

You are a cybersecurity engineer at DataSafe Bank, a financial institution that handles highly sensitive customer data. Your primary responsibility is implementing robust cryptographic solutions to ensure secure communication, data integrity, and confidentiality. This involves leveraging industry-standard cryptographic techniques such as RSA and AES encryption, digital signatures, and Public Key Infrastructure (PKI). These measures are critical for safeguarding customer information, verifying the authenticity of documents, and enabling secure key exchange mechanisms.

All tasks will be accomplished using OpenSSL, a widely recognized and trusted tool for implementing cryptographic protocols and standards. You will play a vital role in maintaining DataSafe Bank's commitment to customer security and trust.

Project instructions

Setting up the system

Before you begin, you must have the following:

- A laptop or desktop to conduct the final project
- Stable internet connection
- Google Chrome pre-installed as a browser

Guidelines for project completion

To ensure a smooth and successful completion of the OpenSSL lab and the associated AI-graded assignment, please adhere to the following guidelines:

- Begin by completing the OpenSSL lab on this CloudIDE platform. Carefully execute all required commands and document the resulting outputs, as these will serve as the inputs for the AI-graded assessment.
- Once you have completed the lab, proceed with the AI-graded assignment. Copy the inputs and outputs of all the lab tasks into a text editor (such as Notepad), compile your responses, and then copy and paste them into the AI platform in one comprehensive submission.
- It is highly recommended that you execute all commands within a single CloudIDE session and complete the AI assignment within one continuous session. This ensures consistency and prevents potential loss of progress.
- Prior to submission, thoroughly review all responses to ensure their accuracy and completeness. Additionally, save your work regularly to prevent any technical disruptions from affecting your progress.

Initial Cloud IDE setup

To complete the project tasks, you are required to download the following sample files to the lab workspace.

1. Use the following command to download the file **contract.txt**

```
wget https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/gGRPyP5ZWcE8XrHBIZDGeg/contract.txt
```

2. Use the following command to download the file **customer-data.txt**

```
wget https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/XA2fYsvuesaeLSmx73pdzQ/customer-data.txt
```

3. Use the following command to download the file **password.txt**

```
wget https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/neZxDIu0eUWyQM8LqoFMbQ/password.txt
```

4. Use the following command to download the file **sensitive-data.txt**

```
wget https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/Jqc3jauW9lIfbLDBS29Q2A/sensitive-data.txt
```

Completing project tasks

You may now begin working on your project tasks.

Here are some commands you will need through this project in addition to the ones provided for the tasks.

1. Use the `ls -l` command to verify that the corresponding output files are generated after executing the command.
2. Use the `cat` command to display the contents of a generated file (e.g. public/private key).
3. Use the `cmp` command to compare two files byte by byte.

It is suggested to save all commands and their outputs in any text editor file as per the task for submission in the AI-based assessment. The output of the commands or any content from the public/private key file should be copied and pasted for this assignment.

[A sample copy is available for reference](#)

Task 1: RSA key generation

Scenario:

As a cybersecurity engineer at DataSafe Bank, you are tasked with securing communication channels between the bank's internal systems and its customers. To achieve this, you must generate RSA keys that will be used for encrypting and decrypting sensitive information transmitted over insecure channels.

Task:

Generate a 2048-bit RSA private key and its corresponding public key. Save the private key as `private_key.pem` and the public key as `public_key.pem`.

Step 1: Generate an RSA private key

Hint:

To generate the RSA private key:

- Use the OpenSSL `genpkey` command.
- Specify the algorithm as RSA and set the key length to 2048 bits.

Command:

```
openssl genpkey -algorithm [ALGORITHM] -out [OUTPUT_FILE] -pkeyopt rsa_keygen_bits:[KEY_LENGTH]
```

Expected output:

- `private_key.pem` file containing the RSA private key.

Action required:

Save the command you used to generate the terminal's private key and confirmation output.

Step 2: Extract the RSA public key from the private key

Hint:

To extract the RSA public key from the private key:

- Use the OpenSSL `rsa` command.
- Use the `-pubout` option to specify that the output should be the public key.

Command:

```
openssl rsa -pubout -in [PRIVATE_KEY_FILE] -out [PUBLIC_KEY_FILE]
```

Expected output:

- `public_key.pem` file containing the RSA public key.

Action required:

Save the command you used to extract the public key and the confirmation output in the terminal.

Task 2: RSA key encryption and decryption

Scenario:

Once the RSA keys have been generated, you must encrypt sensitive customer data (such as account numbers) to ensure secure transmission. The customer data should be encrypted with the public key, and the recipient (bank server) should decrypt it using the private key.

Task:

Using the RSA keys generated in Task 1, encrypt a sensitive customer data file (customer_data.txt) and decrypt it back to its original form.

Step 1: Encrypt customer data using the public key

Hint:

To encrypt the customer data:

- Use the OpenSSL pkeyutl command with the -encrypt option.
- Specify the input file as customer_data.txt and the output file as encrypted_data.bin.
- Use the -pubin option to indicate that the key is a public key.

Command:

```
openssl pkeyutl -encrypt -inkey [PUBLIC_KEY_FILE] -pubin -in [PLAINTEXT_FILE] -out [ENCRYPTED_FILE]
```

Expected output:

- encrypted_data.bin file containing the encrypted data.

Action required:

Save the command you used to encrypt the customer data and the confirmation output in the terminal.

Step 2: Decrypt the data using the private key

Hint:

To decrypt the encrypted data:

- Use the OpenSSL pkeyutl command with the -decrypt option.
- Specify the input file as encrypted_data.bin and the output file as decrypted_data.txt.

Command:

```
openssl pkeyutl -decrypt -inkey [PRIVATE_KEY_FILE] -in [ENCRYPTED_FILE] -out [DECRYPTED_FILE]
```

Expected output:

- decrypted_data.txt file containing the decrypted data should match the original customer_data.txt.

Action required:

Save the command you used to decrypt the customer data and the confirmation output in the terminal.

Task 3: AES encryption

Scenario:

DataSafe Bank must securely store sensitive data (for example, transaction logs, user passwords) in its database. To ensure confidentiality, this data is encrypted using AES encryption before storage. The encrypted data can only be decrypted using the correct AES key.

Task:

Encrypt a sensitive data file (sensitive_data.txt) using AES-256 encryption and decrypt it back using the AES key.

Step 1: Encrypt data using AES-256-CBC

Hint:

To encrypt the data using AES-256 encryption:

- Use the OpenSSL enc command with the -aes-256-cbc option to specify the AES-256-CBC algorithm.
- Use the -salt option to add random data (salt) to make encryption more secure.
- Provide the input file (sensitive_data.txt) and specify the output file (encrypted_data_aes.bin).
- Use a password for encryption with the -pass pass:securepassword option.

Command:

```
openssl enc -[ALGORITHM] -salt -in [INPUT_FILE] -out [ENCRYPTED_FILE] -pass pass:[PASSWORD]
```

Expected output:

- encrypted_data_aes.bin file containing the AES-encrypted data.

Action required:

Save the command you used to encrypt the data and the confirmation output in the terminal.

Step 2: Decrypt data using the AES key

Hint:

To decrypt the data using the AES key:

- Use the OpenSSL enc command with the -d option for decryption.
- Specify the encrypted input file (encrypted_data_aes.bin) and the output file for the decrypted data (decrypted_data_aes.txt).
- Use the same password used for encryption to decrypt the data.

Command:

```
openssl enc -d -[ALGORITHM] -in [ENCRYPTED_FILE] -out [DECRYPTED_FILE] -pass pass:[PASSWORD]
```

Expected output:

- decrypted_data_aes.txt file containing the decrypted data should match the original sensitive_data.txt.

Action required:

Save the command you used to decrypt the data and the confirmation output in the terminal.

Task 4: Digital signature creation

Scenario:

DataSafe Bank needs to ensure the integrity and authenticity of documents (for example, contracts and agreements) transmitted over email. You will create a digital signature using RSA, allowing the recipient to verify the document's integrity and origin.

Task:

Create a digital signature for a document (contract.txt) using your RSA private key. Verify the signature using the corresponding public key.

Step 1: Generate a digital signature for the document

Hint:

To generate a digital signature:

- Use the OpenSSL dgst command with the -sha256 option to specify the SHA-256 hashing algorithm.
- Create the digital signature with the -sign option with the private key.
- Specify the document (contract.txt) and the output file for the signature (document_signature.bin).

Command:

```
openssl dgst -[HASH_ALGORITHM] -sign [PRIVATE_KEY_FILE] -out [SIGNATURE_FILE] [DOCUMENT_FILE]
```

Expected output:

- document_signature.bin file containing the digital signature.

Action required:

Save the command you used to generate the digital signature and the confirmation output in the terminal.

Step 2: Verify the digital signature

Hint:

To verify the digital signature:

- Use the OpenSSL dgst command with the -sha256 option for the SHA-256 algorithm.
- Use the -verify option with the public key to verify the signature.
- Specify the digital signature file (document_signature.bin) and the original document (contract.txt).

Command:

```
openssl dgst -[HASH_ALGORITHM] -verify [PUBLIC_KEY_FILE] -signature [SIGNATURE_FILE] [DOCUMENT_FILE]
```

Expected output:

- Verification result: "Verified OK" or "Verification Failure".

Action required:

Save the command you used to verify the terminal's digital signature and confirmation output.

Task 5: Diffie-Hellman key exchange

Scenario:

DataSafe Bank must establish a secure, shared secret between two parties (an employee and a bank server) without transmitting the secret over the network. To achieve this, the Diffie-Hellman key exchange protocol will be used.

Task:

Implement Diffie-Hellman key exchange to securely exchange keys between two parties (Alice and Bob)

Step 1: Generate DH parameters

Hint:

To generate the Diffie-Hellman parameters:

- Use the OpenSSL dhparam command.
- Specify the key length (2048 bits) to generate secure parameters.

Command:

```
openssl dhparam -out [OUTPUT_FILE] [KEY_LENGTH]
```

Expected outputs:

- dhparam.pem file containing the DH parameters.

Action required:

Save the command you used to generate the DH parameters and the confirmation of output, along with the keys for both parties.

Step 2: Generate Alice's private key

Command:

```
openssl genpkey -paramfile [PARAM_FILE] -out [PRIVATE_KEY_FILE]
```

Expected output:

- File containing Alice's private key'

Action required:

Save the command you used to generate Alice's private key' and the output.

Step 3: Generate Alice's public key

Command:

```
openssl pkey -in [PRIVATE_KEY_FILE] -pubout -out [PUBLIC_KEY_FILE]
```

Expected output:

- File containing Alice's public key'

Action required:

Save of the command you used to generate Alice's public key' and the output.

Step 4: Generate Bob's private key

Command:

```
openssl genpkey -paramfile [PARAM_FILE] -out [PRIVATE_KEY_FILE]
```

Expected output:

- File containing Bobs's private key'and the output.

Action required:

Save command you used to generate Bobs's private key' and the output.

Step 5: Generate Bob's public key

Command:

```
openssl pkey -in [PRIVATE_KEY_FILE] -pubout -out [PUBLIC_KEY_FILE]
```

Expected output:

- File containing Bobs's public key'

Action required:

Save the command you used to generate Bob's public key' and the output.

Step 6: Compute the shared secret for Alice.

Hint:

Compute the shared secret for ALice using her private key and Bob's public key

Commands:**Compute the shared secret for Alice:**

```
openssl pkeyutl -derive -inkey [ALICE_PRIVATE_KEY_FILE] -peerkey [BOB_PUBLIC_KEY_FILE] -out [SHARED_SECRET_FILE]
```

Expected output:

- alice_shared_secret.bin containing Alice's shared secret.

Action required:

Save the command and the output that display Alice's shared secret.

Step 7: Compute the shared secret for Bob

Hint:

Compute the shared secret for Bob using his private key and Alice's public key

Commands:**Compute the shared secret key for Bob:**

```
openssl pkeyutl -derive -inkey [BOB_PRIVATE_KEY_FILE] -peerkey [ALICE_PUBLIC_KEY_FILE] -out [SHARED_SECRET_FILE]
```

Expected output:

- bob_shared_secret.bin file containing Bob's shared secret.

Action required:

Save the command and the output that display Bob's shared secret.

Step 8: Verify the shared secret keys

Hint:

alice_shared_secret.bin and bob_shared_secret.bin files containing the shared secret (both should match).

Expected output:

- `alice_shared_secret.bin` and `bob_shared_secret.bin` files containing the shared secret (both should match).

Action required:

Save the output that displays the shared secret keys. The output should include the successful output showing that both shared secret keys match.

Task 6: Public Key Infrastructure (PKI) setup

Scenario:

DataSafe Bank requires the setup of a Certificate Authority (CA) to sign and issue certificates for internal services and servers. You are tasked with creating a root certificate and signing a server certificate for `datasafebank.com`.

Task:

Set up a Certificate Authority (CA) using OpenSSL. Generate a root certificate and a signed certificate for `datasafebank.com`.

Step 1: Generate the root private key

Hint:

To generate the root private key:

- Use the OpenSSL `genrsa` command.
- Specify the key length (2048 bits) for security.

Command:

- To generate the root private key:

```
openssl genrsa -out [PRIVATE_KEY_FILE] [KEY_LENGTH]
```

Expected output:

- `ca_private_key.pem` file containing the root private key.

Action required:

Copy-paste the command you used to generate the root private key and the confirmation output in the terminal.

Step 2: Create the root certificate

Hint:

- Use the OpenSSL `req` command with the `-x509` option to generate a self-signed certificate.
- Specify the root private key (`ca_private_key.pem`) and set the certificate validity (365 days).

Command:

```
openssl req -x509 -new -key [PRIVATE_KEY_FILE] -out [CERTIFICATE_FILE] -days [DAYS]
```

Expected output:

- `ca_cert.pem` file containing the root certificate.

Action required:

Copy-paste the command you used to create the terminal's root certificate and confirmation output.

Step 3: Generate a CSR for the server

Hint:

To generate the certificate signing request (CSR) for the server:

- Use the OpenSSL req command with the -new option.
- Specify the private key for datasafebank.com and the output CSR file (datasafebank_csr.csr).

Command:

```
openssl req -new -key [PRIVATE_KEY_FILE] -out [CSR_FILE]
```

Expected output:

- datasafebank_csr.csr file containing the server's CSR.

Action required:

Copy-paste of the command you used to generate the CSR and the confirmation output in the terminal.

Step 4: Sign the CSR with the root certificate

Hint:

To sign the CSR using the root certificate:

- Use the OpenSSL x509 command with the -req option to sign the CSR.
- Specify the root certificate (ca_cert.pem) and the root private key (ca_private_key.pem).
- Set the server certificate's validity period (for example , 365 days).

Command:

```
openssl x509 -req -in [CSR_FILE] -CA [CA_CERT_FILE] -CAkey [CA_PRIVATE_KEY_FILE] -CAcreateserial -out [CERTIFICATE_FILE] -days [DAYS]
```

Expected output:

- datasafebank_cert.pem file containing the signed server certificate.

Action required:

Copy-paste the command you used to sign the CSR and the confirmation output in the terminal.

Task 7: Hashing with SHA-256

Scenario:

DataSafe Bank needs to store passwords securely. Instead of storing passwords in plain text, you will hash them using SHA-256 to ensure they are stored securely.

Task:

Hash a password using SHA-256 and verify its integrity.

Step 1: Hash the password using SHA-256

Hint:

To hash the password using SHA-256:

- Use the OpenSSL dgst command with the -sha256 option to create a SHA-256 hash.
- Specify the input password file (password.txt) and the output file for the hashed password (hashed_password.txt).

Command:

```
openssl dgst -sha256 -out [OUTPUT_FILE] [INPUT_FILE]
```

Expected output:

- hashed_password.txt file containing the SHA-256 hash of the password.

Action required:

Copy-paste the command you used to hash the password and the confirmation output in the terminal.

Step 2: Verify the hash

Hint:

To verify the integrity of the hashed password:

- Re-hash the original password file using the openssl dgst command with the -sha256 option.
- Compare the output hash directly with the contents of hashed_password.txt.

Command:

```
openssl dgst -sha256 [INPUT_FILE]
```

Manually compare the output with the contents of hashed_password.txt. They should be identical.

Expected output:

- The original hash output should match the value stored in hashed_password.txt.

Action required:

Copy-paste the command you used to verify the hash and the terminal output showing the match.

Task 8: Digital certificate generation

Scenario:

DataSafe Bank needs to issue digital certificates to its employees and internal systems for secure communication. You will use OpenSSL to generate a CSR (Certificate Signing Request) and sign it with the root certificate.

Task:

Create a CSR for a server and generate a self-signed certificate.

Step 1: Generate a private key for the server

Hint:

To generate the private key for the server:

- Use the OpenSSL genpkey command with the -algorithm RSA option.
- Specify the key size (2048 bits).

Command:

```
openssl genpkey -algorithm [ALGORITHM] -out [PRIVATE_KEY_FILE] -pkeyopt [KEY_LENGTH_OPTION]
```

Expected output:

- server_private_key.pem file containing the private key for the server.

Action required:
Copy-paste the command you used to generate the server's private key and the confirmation output in the terminal.

Step 2: Create a CSR

Hint:
To create the CSR for the server:

- Use the OpenSSL req command with the -new option.
- Specify the private key (server_private_key.pem) and the output CSR file (server_csr.csr).

Command:

```
openssl req -new -key [PRIVATE_KEY_FILE] -out [CSR_FILE]
```

Expected output:

- server_csr.csr file containing the server's CSR.

Action required:
Copy-paste the command you used to create the CSR and the confirmation output in the terminal.

Step 3: Create a self-signed certificate

Hint:
To create a self-signed certificate:

- Use the OpenSSL x509 command with the -req option to sign the CSR using the server's private key.
- Specify the certificate's validity period (for example, 365 days).

Command:

```
openssl x509 -req -in [CSR_FILE] -signkey [PRIVATE_KEY_FILE] -out [CERTIFICATE_FILE] -days [DAYS]
```

Expected output:

- server_cert.pem file containing the self-signed certificate.

Action required:
Copy-paste the command you used to create the self-signed certificate and the confirmation output in the terminal.

Review Criteria

A total of 40 points are allotted for this project.
Your grade will be based on the following distribution of points:

Task	Step	Submission Requirement	Points
1	1	Command and output for generating the private key	2
	2	Command and output for extracting the public key	2
2	1	Command and output for encrypting customer data using the public key	2
	2	Command and output for decrypting customer data using the private key	2
3	1	Command and output for encrypting sensitive data using AES-256	2

Task	Step	Submission Requirement	Points
	2	Command and output for decrypting data back to its original form	2
4	1	Command and output for creating a digital signature for a document	2
	2	Command and output for verifying the integrity of the digital signature	2
5	1	Command to generate Diffie-Hellman parameters	1
	2	Command to generate Alice's private key	1
	3	Command to generate Alice's public key	1
	4	Command to generate Bob's private key	1
	5	Command to generate Bob's public key	1
	6	Command to compute Alice's shared secret	1
	7	Command to compute Bob's shared secret	1
	8	Command to compare the shared secret keys and output verifying the matching shared secret	1
6	1	Command to generate the root private key	1
	2	Command to create the root certificate	1
	3	Command and output for generating a CSR for datasafebank.com	2
	4	Command and output for signing the CSR with the root certificate	2
7	1	Command and output for hashing a password using SHA-256	2
	2	Command and output for verifying the integrity of the hash	2
8	1	Command and output for generating a private key for the server	2
	2	Command and output for creating a CSR	2
	3	Command and output for creating a self-signed certificate	2
		Total	40

Submission and evaluation

Submitting your project

Check your submissions thoroughly and ensure they are ready to be submitted for review. Copy-paste your outputs on any text editor and paste them into to Coursera platform for easy of submission to avoid moving between screens.

Author

Dr. Manish Kumar

© IBM Corporation. All rights reserved.